

```
<!doctype html>
<html lang="en">
<head>
  <meta charset="utf-8">
  <title>Student Portfolio</title>
</head>
<body>
  <header><h1>Hello, Web</h1></header>
  <main>
    <section id="about">
      <p>Structure. Meaning. Accessibility.</p>
    </section>
  </main>
</body>
</html>
```

<!-- HTML5 --> structure + meaning -->

Learn more
ZERO TO PROFESSIONAL — FULL COURSE NOTES
<input type="email" required>

```
<picture><source srcset="a.jpg"></picture></doctype html>
<html lang=
<head>
  <meta charset="utf-8">
  <title>Stu
</head>
<body>
  <header><h1>Hello, Web</h1></header>
  <main>
    <section id="about">
      <p>Structure. Meaning. Accessibility.</p>
    </section>
  </main>
</body>
</html>
```

Complete HTML5 Mastery Guide

A structured, project-based path from your very first tag to accessible, secure, SEO-ready, production-grade markup — with authentic references to the official specifications and documentation used by professional developers.

HTML5 STRUCTURE

CSS PRESENTATION

JS BEHAVIOR

ACCESSIBLE & SEMANTIC

<!-- HTML5 --> structure + meaning -->

Learn more

<input type="email" required>

<picture><source srcset="a.jpg"></picture></doctype html>

<html lang="en">

<head>

<meta charset="utf-8">

<title>Student Portfolio</title>

</head>

<body>

<header><h1>Hello, Web</h1></header>

<main>

<section id="about">

<p>Structure. Meaning. Accessibility.</p>

</section>

</main>

</body>

</html>

<!-- HTML5 --> structure + meaning -->

Learn more

31+ Chapters
Fundamentals to full projects

Official Sources
WHATWG · MDN · W3C · WAI

Method
Read → Type → Run → Inspect → Modify

Welcome — How This Guide Works

This edition has been redesigned end-to-end: every chapter is expanded with deeper explanations, additional real-world examples, styled code previews, accessibility and security notes, and links to the authentic official documentation (WHATWG Living Standard, MDN Web Docs, W3C, and Google Search Central) so you can always verify details at the source.

1

READ THE
CONCEPT

2

TYPE THE
EXAMPLE

3

RUN IN BROWSER

4

INSPECT THE
RESULT

5

MODIFY &
EXPERIMENT

Layer	Responsibility	Typical Job	Official Spec
HTML	Structure & meaning	Headings, forms, landmarks, images	html.spec.whatwg.org
CSS	Presentation	Color, layout, spacing, typography	w3.org/Style/CSS
JavaScript	Behavior & logic	Events, DOM updates, API calls	tc39.es/ecma262

Table of Contents

PART I — FOUNDATIONS

01 HTML Fundamentals

02 Syntax & Terminology

03 Document Structure

04 Head & Metadata

05 Global Attributes

PART II — CONTENT

06 Text, Headings & Formatting

07 Links & URLs

08 Images & Responsive Images

09 Audio & Video

10 Lists

11 Tables

PART III — FORMS

12 Forms

13 Input Types

14 Validation

PART IV — STRUCTURE & MEANING

15 Semantic HTML

16 Layout, Div & Span

17 Iframes & Embedding

18 Entities & File Paths

PART V — QUALITY & REACH

19 Accessibility

20 Responsive HTML

21 Security Fundamentals

22 SEO

PART VI — INTEGRATION

23 HTML + CSS + JavaScript

24 Browser & Web APIs

25 Deprecated / Obsolete HTML

26 Common Mistakes

PART VII — PRACTICE

27 Complete Website Example

28 Mini Projects

29 HTML5 Cheat Sheet

30 Practice Q&A

31 Final Checklist

APPENDIX

A HTML5 Timeline & History

B Official Resources & Further Reading



HTML (HyperText Markup Language) is the standard markup language for structuring content on the web. It is **not** a general-purpose programming language — it has no variables, loops, or functions of its own. Its entire job is to describe *what a piece of content is*, so browsers, search engines, screen readers, and other software can interpret it correctly.

HyperText means documents and resources are connected by links, letting users jump between them.

Markup means the content is annotated with elements that describe structure and meaning — a paragraph, a heading, a list item, a form field — rather than describing how it should look.

Technology	Responsibility	Typical Examples
HTML	Structure / meaning	Headings, forms, images, landmarks
CSS	Presentation	Color, layout, spacing, animation
JavaScript	Behavior / logic	Events, DOM updates, API calls

When a browser loads a page, it parses the HTML into an in-memory tree structure called the **DOM** (Document Object Model). CSS selectors match nodes in that tree to style them, and JavaScript can read, create, or mutate that same tree to make the page interactive.

```
portfolio.html HTML
<h1>Student Portfolio</h1>
<p>Information Technology student.</p>
<a href="/projects">View projects</a>
```



Why the distinction matters

Writing a `<div>` styled to look like a heading instead of using `<h1>`–`<h6>` may look identical visually, but it breaks meaning: screen readers, search engines, and browser outline tools all rely on real semantic elements, not visual mimicry.

A Brief History, in One Paragraph

HTML was created by Tim Berners-Lee in 1991 at CERN. It was standardized through several numbered versions (HTML 2 → 4.01) and an abandoned XHTML 2 effort, before the WHATWG and W3C converged on **HTML5**, first published as a W3C Recommendation in 2014. Since 2019, the WHATWG's **HTML Living Standard** is the single, continuously-updated authoritative specification that both the W3C and all major browser vendors follow — there is no more "HTML6" version number; the spec simply evolves.

WHATWG HTML Living Standard (official spec)

<https://html.spec.whatwg.org/multipage/>

The authoritative, continuously updated HTML specification maintained by browser vendors.

MDN — HTML: HyperText Markup Language

<https://developer.mozilla.org/en-US/docs/Web/HTML>

Mozilla's developer-friendly HTML guides, tutorials, and full reference.

web.dev — Learn HTML

<https://web.dev/learn/html>

Google's structured, beginner-to-advanced HTML learning path.

Term	Definition
Element	The complete structure — opening tag, content, and closing tag together.
Tag	The markup notation itself, e.g. <code><p></code> or <code></p></code> .
Attribute	Additional information placed inside the opening tag, e.g. <code>class="intro"</code> .
Content	The text or nested elements an element contains.
Nesting	Placing elements inside other elements to build a tree.
Void element	An element that cannot contain children and is never closed, e.g. <code></code> .

```
syntax.html HTML
<p class="intro">Hello world.</p>
<a href="https://example.com">Visit</a>

```

Void (Self-Closing) Elements

These elements have no closing tag and cannot wrap content:

`img` `br` `hr` `input` `meta` `link` `area` `base` `col` `embed` `source` `track` `wbr`

Comments

```
comments.html HTML
<!-- This is a comment, never rendered by the browser -->
```



Never hide secrets in HTML comments

Anyone can open "View Page Source" or DevTools. API keys, internal notes, or unpublished URLs left in comments are effectively public.

Well-Formed Markup Rules

- ☐ Nest elements correctly — close the innermost tag first.
- ☐ Always quote attribute values: `class="card"`, not `class=card`.
- ☐ Lowercase tag and attribute names by convention (HTML itself is case-insensitive).
- ☐ One root `<html>` element per document.
- ☐ Every non-void element that is opened must eventually be closed.

WHATWG — Syntax section of the Living Standard

<https://html.spec.whatwg.org/multipage/syntax.html>

The formal parsing and syntax rules browsers implement.

MDN — Getting started with HTML

https://developer.mozilla.org/en-US/docs/Learn/HTML/Introduction_to_HTML/Getting_started

A gentle, official walkthrough of HTML terminology for beginners.



The skeleton every HTML page starts from

```
index.html HTML

<!doctype html>
<html lang="en">
<head>
  <meta charset="utf-8">
  <meta name="viewport"
    content="width=device-width, initial-scale=1">
  <title>My Website</title>
</head>
<body>
  <header><h1>My Website</h1></header>
  <main><p>Hello!</p></main>
  <footer><p>Copyright</p></footer>
</body>
</html>
```

- **<!doctype html>**
Requests standards mode. Without it, browsers may fall back to "quirks mode," which resurrects historic rendering bugs for backward compatibility.
- **<html lang="en">**
The root element of the document. `lang` declares the page's language, which helps screen readers choose the right pronunciation and helps browsers offer translation.
- **<head>**
Contains metadata and resource links — nothing inside it is rendered directly as visible page content.
- **<body>**
Contains everything the user actually sees and interacts with.



Indentation is a courtesy, not a rule

Indentation only helps humans read the nesting; it has zero effect on rendering. Consistent indentation, however, dramatically reduces bugs during editing.

MDN — The basic structure of an HTML page

https://developer.mozilla.org/en-US/docs/Learn/HTML/Introduction_to_HTML/Document_and_website_structure

Explains doctype, html, head, and body with live examples.

WHATWG — The html element

<https://html.spec.whatwg.org/multipage/semantics.html#the-html-element>

Formal definition of the root element and document structure.



Element	Purpose	Example
<code>title</code>	Document / browser-tab title	<code><title>Home</title></code>
<code>meta</code>	Metadata key/value	<code><meta charset="utf-8"></code>
<code>link</code>	External resource	<code><link rel="stylesheet" href="style.css"></code>
<code>style</code>	Embedded CSS	<code><style>...</style></code>
<code>script</code>	JavaScript	<code><script src="app.js" defer></code>
<code>base</code>	Base URL for relative links	<code><base href="https://example.com/"></code>

```
head.html HTML
<head>
  <meta charset="utf-8">
  <meta name="viewport" content="width=device-width, initial-scale=1">
  <meta name="description" content="Learn HTML step by step.">
  <title>HTML Course</title>
  <link rel="stylesheet" href="styles.css">
  <script src="app.js" defer></script>
</head>
```

Key Meta Tags Explained

Name	What it does
<code>charset="utf-8"</code>	UTF-8 is the standard modern encoding, covering virtually every language and emoji.
<code>viewport</code>	Tells mobile browsers to use the device's actual width instead of faking a desktop-sized page.
<code>description</code>	Search engines often show this text as the snippet under your page's title in results.
<code>theme-color</code>	Tints the browser UI (address bar) on supporting mobile browsers.
<code>robots</code>	Controls whether crawlers should index the page and follow its links.



defer vs async

`defer` downloads the script in parallel with HTML parsing and runs it only after parsing finishes, in document order. `async` also downloads in parallel but runs the script the instant it is ready, potentially before parsing finishes and out of order relative to other scripts. For most page scripts, `defer` is the safer default.

MDN — The head metadata in HTML

https://developer.mozilla.org/en-US/docs/Learn/HTML/Introduction_to_HTML/The_head_metadata_in_HTML

Official guide to everything that belongs inside the head element.

MDN — meta element reference

<https://developer.mozilla.org/en-US/docs/Web/HTML/Element/meta>

Full list of standard and non-standard meta names.

Google Search Central — meta tags

<https://developers.google.com/search/docs/crawling-indexing/special-tags>

Which meta tags Google actually reads and how.



Attribute	Purpose	Example
<code>id</code>	Unique identifier in the document	<code>id="contact"</code>
<code>class</code>	Reusable grouping for CSS/JS	<code>class="card"</code>
<code>title</code>	Advisory tooltip text	<code>title="Details"</code>
<code>lang</code>	Language of the element's content	<code>lang="ur"</code>
<code>dir</code>	Text direction	<code>dir="rtl"</code>
<code>hidden</code>	Currently not relevant / removed from render	<code>hidden</code>
<code>tabindex</code>	Keyboard focus order	<code>tabindex="0"</code>
<code>style</code>	Inline CSS (use sparingly)	<code>style="color:red"</code>
<code>data-*</code>	Custom application data	<code>data-id="42"</code>
<code>contenteditable</code>	Makes content directly editable	<code>contenteditable="true"</code>
<code>draggable</code>	Enables the HTML Drag and Drop API	<code>draggable="true"</code>
<code>spellcheck</code>	Requests spellcheck UI	<code>spellcheck="true"</code>

global-attrs.html

HTML

```
<div id="pricing-card" class="card featured" data-plan="pro" tabindex="0">
  <p title="Billed annually">Pro Plan</p>
</div>
```



id vs class

IDs must be unique per document and are used for a single specific element (fragment links, form label pairing, JS lookups). Classes can repeat across many elements and are the normal way to apply shared styling.

MDN — Global attributes

https://developer.mozilla.org/en-US/docs/Web/HTML/Global_attributes

Complete authoritative list of attributes valid on every element.



`h1` through `h6` represent a heading hierarchy — they describe document outline and importance, not font size. Use CSS to control visual size; never pick a heading level purely because of how big it looks.

```
headings.html HTML
<h1>Web Development</h1>
<h2>HTML</h2>
<h3>Forms</h3>
<p>A paragraph.</p>
<p>Line one<br>Line two</p>
<hr>
```

Element	Meaning
<code>strong</code>	Strong importance
<code>em</code>	Stress emphasis
<code>b</code>	Attention, without implying strong importance
<code>i</code>	Alternate voice, mood, or technical term
<code>mark</code>	Relevant highlight (e.g. search-term match)
<code>small</code>	Side comments / fine print
<code>del</code> / <code>ins</code>	Deleted / inserted content
<code>sub</code> / <code>sup</code>	Subscript / superscript
<code>code</code>	Inline code fragment
<code>kbd</code>	Keyboard input
<code>samp</code>	Sample program output
<code>var</code>	A variable name
<code>abbr</code>	Abbreviation (pairs with the title attribute)
<code>cite</code>	Title of a referenced work
<code>blockquote</code> / <code>q</code>	Long / short quotation
<code>pre</code>	Preformatted, whitespace-preserving text



br is not a layout tool

Use the `br` element only for a genuine, meaningful line break (like in a postal address or a poem). Never chain several `br` tags to create vertical spacing — that is CSS's job (margin/padding).

MDN — HTML text fundamentals

https://developer.mozilla.org/en-US/docs/Learn/HTML/Introduction_to_HTML/HTML_text_fundamentals

Headings, paragraphs, lists, emphasis and quotations explained together.

MDN — Element reference: Inline text semantics

https://developer.mozilla.org/en-US/docs/Web/HTML/Element#inline_text_semantics

Full catalogue of inline meaning-carrying elements.



links.html

HTML

```
<a href="https://example.com">Absolute URL</a>
<a href="/about.html">Root-relative URL</a>
<a href="images/photo.jpg">Relative URL</a>
<a href="#contact">Fragment</a>
<a href="mailto:hello@example.com">Email</a>
<a href="tel:+923001234567">Phone</a>
<a href="https://example.com" target="_blank" rel="noopener noreferrer">
  Opens safely in a new tab
</a>
```

URL Part	Example	Meaning
Scheme	<code>https</code>	Access protocol
Host	<code>example.com</code>	Server hostname
Port	<code>443</code>	Optional transport port
Path	<code>/about</code>	Resource location on the server
Query	<code>?page=2</code>	Key/value parameters
Fragment	<code>#contact</code>	Client-side target, not normally sent to the server

**target="_blank" needs rel="noopener";**

Without rel="noopener", the page you open can access window.opener and redirect your original tab — a real phishing technique called “tabnabbing.” Add noreferrer too if you also want to withhold the referrer header.

MDN — The a (anchor) element

<https://developer.mozilla.org/en-US/docs/Web/HTML/Element/a>

Full reference for hyperlinks, including rel and target values.

WHATWG — Link types

<https://html.spec.whatwg.org/multipage/links.html#linkTypes>

Every valid value for the rel attribute, defined by the spec.



basic-image.html

HTML

```

```

`src` identifies the image file. `alt` provides the text alternative shown when the image fails to load and read aloud by screen readers. `width` / `height` let the browser reserve space before the image downloads, preventing layout shift. `loading=lazy` defers off-screen images until they're about to enter the viewport.

srcset & sizes

responsive-image.html

HTML

```

```

`srcset` lists image candidates with their intrinsic widths; `sizes` tells the browser how wide the image will be displayed at different viewport widths, so it can pick the smallest sufficient file.

picture — Art Direction & Format Fallback

art-direction.html

HTML

```
<picture>
  <source media="(max-width: 600px)" srcset="mobile.jpg">
  <source type="image/avif" srcset="desktop.avif">
  
</picture>
```

alt Guidance

Rule

Meaningful image	Write a concise, descriptive alt text conveying its purpose.
Decorative image	Use <code>alt=""</code> so screen readers skip it silently.
Image duplicating nearby text	Use <code>alt=""</code> to avoid repeating the same information twice.
Image that is also a link	Describe the link's destination/action, not just the picture.



Modern formats

Prefer AVIF or WebP over JPEG/PNG where possible — typically 25–50% smaller at equivalent visual quality. Use `picture` with type-based source fallbacks for browsers that don't support the newer format.

https://developer.mozilla.org/en-US/docs/Web/HTML/Guides/Responsive_images

Deep dive into srcset, sizes and picture with worked examples.

MDN — img element reference

<https://developer.mozilla.org/en-US/docs/Web/HTML/Element/img>

Every attribute of the img element, including loading and decoding.

web.dev — Image optimization guides

<https://web.dev/learn/images/>

Google's practical guide to formats, compression and responsive delivery.



```
audio.html HTML
<audio controls>
  <source src="lesson.mp3" type="audio/mpeg">
  <source src="lesson.ogg" type="audio/ogg">
  Your browser doesn't support the audio element.
</audio>
```

```
video.html HTML
<video controls width="640" poster="poster.jpg" preload="metadata">
  <source src="lesson.mp4" type="video/mp4">
  <track src="captions-en.vtt" kind="captions" srclang="en" label="English" default>
  <track src="captions-ur.vtt" kind="captions" srclang="ur" label="Urdu">
</video>
```

Attribute	Effect
<code>controls</code>	Shows the browser's built-in play/pause/volume UI.
<code>autoplay</code>	Starts playback automatically (most browsers block audible autoplay).
<code>muted</code>	Required by most browsers to allow autoplay at all.
<code>loop</code>	Restarts playback automatically when it ends.
<code>poster</code>	Image shown before the video starts playing.
<code>preload</code>	Hints how much to buffer ahead: none, metadata, or auto.



Always caption your video

The track element with `kind=captions` lets Deaf and hard-of-hearing users, and anyone watching muted, follow along. It also gives search engines readable text content.

MDN — Video and audio content

https://developer.mozilla.org/en-US/docs/Learn/HTML/Multimedia_and_embedding/Video_and_audio_content
Official tutorial covering audio, video, and track elements.

WHATWG — Media elements

<https://html.spec.whatwg.org/multipage/media.html>
Formal specification of audio, video, source, and track.



Element	Purpose
<code>ul</code>	Unordered (bulleted) list
<code>ol</code>	Ordered (numbered) list
<code>li</code>	A list item
<code>dl</code>	Description list
<code>dt</code>	Term being described
<code>dd</code>	The description or details

```
lists.html HTML
<ul>
  <li>HTML</li>
  <li>CSS</li>
</ul>

<ol start="3" reversed>
  <li>Create file</li>
  <li>Write HTML</li>
</ol>

<dl>
  <dt>HTML</dt>
  <dd>Structures content.</dd>
  <dt>CSS</dt>
  <dd>Styles content.</dd>
</dl>
```



Nesting lists for menus and sitemaps

A nav element commonly wraps a ul of links; nested ul elements inside li items create sub-menus, which map naturally to how screen readers announce list depth.

MDN — HTML lists

https://developer.mozilla.org/en-US/docs/Learn/HTML/Introduction_to_HTML/HTML_text_fundamentals#lists

Ordered, unordered and description lists with examples.



```
table.html HTML

<table>
  <caption>Student Results</caption>
  <thead>
    <tr><th scope="col">Name</th><th scope="col">CGPA</th></tr>
  </thead>
  <tbody>
    <tr><td>Ali</td><td>3.7</td></tr>
    <tr><td>Sara</td><td>3.9</td></tr>
  </tbody>
  <tfoot>
    <tr><td>Average</td><td>3.8</td></tr>
  </tfoot>
</table>
```

Element	Purpose
<code>table</code>	Container for the whole table
<code>caption</code>	Title/description of the table
<code>thead</code> / <code>tbody</code> / <code>tfoot</code>	Row groups (header, body, footer)
<code>tr</code>	A table row
<code>th</code>	A header cell
<code>td</code>	A data cell
<code>colgroup</code> / <code>col</code>	Column-level styling definitions

**scope="col" / scope="row";**

These communicate header-to-cell relationships to assistive technology, so a screen reader can announce "Ali, Name: Ali, CGPA: 3.7" correctly rather than reading a flat, disconnected grid of text.

MDN — HTML table basics

<https://developer.mozilla.org/en-US/docs/Learn/HTML/Tables/Basics>

Building accessible, well-structured tables step by step.

WAI — Tables Tutorial

<https://www.w3.org/WAI/tutorials/tables/>

W3C Web Accessibility Initiative's official tables accessibility tutorial.



form.html

HTML

```
<form action="/register" method="post">
  <label for="name">Name</label>
  <input id="name" name="name" type="text" required>

  <label for="email">Email</label>
  <input id="email" name="email" type="email" required>

  <button type="submit">Create account</button>
</form>
```

Concept	Meaning
<code>action</code>	URL the form data is submitted to
<code>method</code>	HTTP method — usually GET or POST
<code>name</code>	Field name key used in the submitted data
<code>value</code>	The field's current/submitted value
<code>label</code>	Accessible, clickable name for a control
<code>button</code>	Action or submission control
<code>fieldset / legend</code>	Groups related controls with a caption



GET vs POST

GET commonly appends data to the URL as a query string (visible, bookmarkable, size-limited, cacheable) — good for searches. POST commonly sends data in the request body (hidden from the URL, no practical size limit) — required for anything sensitive or that changes server state. HTTPS is mandatory for any form handling sensitive data.

MDN — Your first HTML form

https://developer.mozilla.org/en-US/docs/Learn/Forms/Your_first_form

Official step-by-step introduction to building forms.

WHATWG — Forms

<https://html.spec.whatwg.org/multipage/forms.html>

The complete forms specification, including submission algorithms.

Type	Typical Use
<code>text</code>	General free text
<code>password</code>	Masked password entry
<code>email</code>	Email address, with built-in format check
<code>url</code>	Web address, with built-in format check
<code>tel</code>	Telephone number (no built-in format check)
<code>search</code>	Search box (may show a clear button)
<code>number</code>	Numeric spinner
<code>range</code>	Slider between min and max
<code>date</code>	Date picker
<code>month</code>	Month/year picker
<code>week</code>	Week/year picker
<code>time</code>	Time picker
<code>datetime-local</code>	Local date and time picker
<code>color</code>	Native color picker
<code>checkbox</code>	Independent yes/no choice
<code>radio</code>	One choice within a named group
<code>file</code>	File selection
<code>hidden</code>	Value submitted but not shown (never for secrets)
<code>submit</code>	Submits the form
<code>reset</code>	Resets fields to defaults
<code>range</code> / <code>image</code>	Slider / image-based submit button

```
input-types.html HTML
<input type="email" name="email" required>
<input type="number" name="age" min="18" max="100">
<input type="date" name="dob">
<input type="checkbox" name="terms" required>
<input type="radio" name="plan" value="basic" id="basic">
<input type="radio" name="plan" value="pro" id="pro">
<input type="file" name="resume" accept=".pdf,.doc,.docx">
```



Radio groups share a name

Radio buttons only behave as a mutually-exclusive group when they share the same name attribute. Checkboxes, by contrast, are always independent, even when grouped visually.

MDN — input types reference

<https://developer.mozilla.org/en-US/docs/Web/HTML/Element/input>

Every input type with browser support notes and examples.



HTML provides free, built-in **constraint validation**, but it is a convenience for users, not a security boundary. Browser validation can be bypassed entirely (disabled JS, direct API calls, curl). The server **must** independently validate and authorize every submitted value.

Attribute	Purpose
<code>required</code>	Field must have a value before submission
<code>minlength</code> / <code>maxlength</code>	Constrains text length
<code>min</code> / <code>max</code>	Constrains numeric or date bounds
<code>step</code>	Allowed increments for number/range/date
<code>pattern</code>	Custom regular-expression constraint
<code>multiple</code>	Allows multiple values (email, file)
<code>accept</code>	Hints acceptable file types to the file picker
<code>autocomplete</code>	Hints the browser's autofill behavior

validation.html

HTML

```
<input name="username" minlength="3" maxlength="20" required>  
<input name="age" type="number" min="18" max="100" required>  
<input name="postcode" pattern="[0-9]{5}" title="Five digit postcode">
```

**Never trust hidden fields**

A hidden input holding a price, role, or permission level can be edited by anyone using their browser's DevTools before submission. Compute and check trusted values server-side, always.

MDN — Client-side form validation

https://developer.mozilla.org/en-US/docs/Learn/Forms/Form_validation

Explains built-in validation attributes and the Constraint Validation API.



Element	Meaning
<code>header</code>	Introductory content for a page or section
<code>nav</code>	A block of navigation links
<code>main</code>	The dominant, unique content of the page (one per page)
<code>section</code>	A thematic grouping of content, usually with a heading
<code>article</code>	Self-contained content that could stand alone (a post, a card)
<code>aside</code>	Content tangentially related to the main content
<code>footer</code>	Footer for a page or section
<code>figure</code> / <code>figcaption</code>	Self-contained media with an optional caption
<code>details</code> / <code>summary</code>	A native, no-JS disclosure widget
<code>time</code>	A machine-readable date/time
<code>dialog</code>	A native modal or non-modal dialog box

```
semantic.html HTML
<details>
  <summary>What browsers support HTML5?</summary>
  <p>All modern evergreen browsers: Chrome, Firefox, Safari, Edge.</p>
</details>

<figure>
  
  <figcaption>Fig 1. Revenue grew 18% quarter over quarter.</figcaption>
</figure>

<p>Published <time datetime="2026-09-22">September 22, 2026</time></p>
```



Meaning first, appearance second

Choose an element because its meaning fits the content. A details element can be restyled with CSS to look like anything — but it will always behave correctly for keyboard and screen-reader users, for free, because the browser implements its behavior natively.

MDN — HTML: A good basis for accessibility

<https://developer.mozilla.org/en-US/docs/Learn/Accessibility/HTML>

Why semantic HTML is the foundation of accessible sites.

WHATWG — Sections

<https://html.spec.whatwg.org/multipage/sections.html>



```
layout.html HTML

<body>
  <header>
    <h1>My Site</h1>
    <nav aria-label="Main navigation">
      <a href="/">Home</a>
      <a href="/about">About</a>
    </nav>
  </header>
  <main>
    <article>
      <h2>HTML Course</h2>
      <section><h3>Intro</h3><p>HTML structures content.</p></section>
    </article>
    <aside><h2>Related</h2></aside>
  </main>
  <footer><p>Copyright</p></footer>
</body>
```

`div` is a generic block-level container with no inherent meaning. `span` is its inline equivalent. Reach for them **only** when no more meaningful semantic element fits — they are the correct tool for pure styling/scripting hooks, not for content that has real structural meaning.

```
card.html HTML

<div class="card">
  <h2>Product</h2>
  <p>Price: <span class="price">$50</span></p>
</div>
```

**div-itis**

A page built entirely from nested divs (“div soup”) loses its document outline: screen reader users lose landmark navigation, and search engines lose structural signals. Reach for header/nav/main/section/article/aside/footer first.

MDN — div element

<https://developer.mozilla.org/en-US/docs/Web/HTML/Element/div>

When and how to use the generic block container.

MDN — span element

<https://developer.mozilla.org/en-US/docs/Web/HTML/Element/span>

When and how to use the generic inline container.



iframe.html

HTML

```
<iframe
  src="https://example.com/widget"
  title="Weather widget"
  width="800" height="500"
  loading="lazy"
  sandbox="allow-scripts">
</iframe>
```

Attribute	Purpose
<code>src</code>	URL of the embedded page
<code>title</code>	Required accessible name describing the embedded content
<code>loading</code>	<code>lazy</code> defers loading until near the viewport
<code>sandbox</code>	Restricts the embedded page's capabilities (scripts, forms, popups)
<code>allow</code>	Grants specific permissions (camera, fullscreen, etc.)



Third-party iframes are a real attack surface

Embedded content can run its own scripts, track users, or attempt clickjacking. Use sandbox to restrict capabilities, and only embed sources you trust.

Prefer a purpose-built element when it fits: `img`, `audio`, or `video` accurately describe their content and are lighter-weight than a full embedded browsing context.

MDN — iframe element

<https://developer.mozilla.org/en-US/docs/Web/HTML/Element/iframe>

Full attribute reference including sandbox values.



Character	Entity
<	<
>	>
&	&
"	"
©	©
non-breaking space	
—	—

To display literal markup as visible text (like in this very course), wrap the reserved characters in character references such as `<` and `>`.

Path Type	Example	Meaning
Same directory	<code>photo.jpg</code>	Current directory
Child directory	<code>images/photo.jpg</code>	Inside a subfolder
Parent directory	<code>../photo.jpg</code>	One level up
Root-relative	<code>/images/photo.jpg</code>	From the site's domain root
Absolute	<code>https://cdn.example.com/a.jpg</code>	A complete, fully qualified URL



A filesystem path is not a URL

Browsers request URLs, not raw operating-system paths. A local Windows path such as `C:/Users/me/photo.jpg` means nothing to a visitor's browser once deployed.

MDN — Entity reference

<https://developer.mozilla.org/en-US/docs/Glossary/Entity>

What character entities are and why they exist.

WHATWG — Named character references

<https://html.spec.whatwg.org/multipage/named-characters.html>

The complete official list of every named HTML entity.

Accessibility (often abbreviated **a11y**) means content works for keyboard users, screen-reader users, users with low vision, motor impairments, or cognitive differences — not as an afterthought, but as a baseline requirement.

- ☐ Use semantic HTML before reaching for ARIA.
- ☐ Always set the correct lang attribute on the html element.
- ☐ Give informative images useful alt text; decorative images use alt="".
- ☐ Label every form control with a real label element.
- ☐ Use buttons for actions and links for navigation — never the reverse.
- ☐ Maintain a logical, unbroken heading hierarchy.
- ☐ Keep every interactive element keyboard-operable (Tab, Enter, Space).
- ☐ Never remove the visible focus outline without providing an equally visible replacement.
- ☐ Provide captions and transcripts for audio/video content.
- ☐ Mark table headers correctly with th and scope.

**Native HTML is already accessible**

A real button element already has the correct role, is focusable, and responds to both Enter and Space out of the box. A styled div pretending to be a button requires you to manually reimplement all of that with ARIA and JavaScript — and it's easy to miss a case.

W3C WAI — Web Accessibility Initiative

<https://www.w3.org/WAI/>

The W3C's home for accessibility standards, guidelines, and tutorials.

WAI-ARIA Authoring Practices Guide (APG)

<https://www.w3.org/WAI/ARIA/apg/>

Official patterns for building accessible custom widgets.

MDN — Accessibility

<https://developer.mozilla.org/en-US/docs/Web/Accessibility>

MDN's complete accessibility learning area.



viewport.html

HTML

```
<meta name="viewport"
      content="width=device-width, initial-scale=1">
```

Responsive design is mostly CSS's job (media queries, flexible grids), but HTML must lay the groundwork with a flexible, semantic structure and responsive assets.

- ☐ Use semantic structure so content reflows logically at any width.
- ☐ Use srcset/sizes (or picture) for responsive image candidates.
- ☐ Avoid layout tables — they don't reflow.
- ☐ Use scalable media (max-width: 100% in CSS).
- ☐ Make every control usable on both touch and keyboard.



The viewport tag alone doesn't make a site responsive

It only stops mobile browsers from faking a 980px-wide desktop viewport. The actual adapting is done by your CSS media queries and flexible layout (Flexbox/Grid).

MDN — Responsive design

https://developer.mozilla.org/en-US/docs/Learn/CSS/CSS_layout/Responsive_Design

How HTML and CSS work together to adapt to viewport size.

web.dev — Responsive Web Design Basics

<https://web.dev/responsive-web-design-basics/>

Google's practical guide to viewport, media queries and flexible content.



- ❑ Never put secrets in HTML — client-delivered content can always be inspected.
- ❑ Hidden inputs are not secure and can be freely modified by the client.
- ❑ Use HTTPS for any page that transmits protected data.
- ❑ Validate and authorize every input on the server, not just the client.
- ❑ Escape untrusted output when generating HTML dynamically, to prevent XSS.
- ❑ Use secure, HttpOnly, and SameSite cookie attributes where appropriate.
- ❑ Review third-party scripts and embedded iframes before trusting them.

```
insecure-example.html HTML
<!-- NOT a secure price store -- a user can edit this value in DevTools -->

```



A hidden field is not a lock

A user can open DevTools, change value=10 to value=0.01, and submit the form. The server must independently determine and verify the trusted price, role, or permission — never trust what the client sends back.



Cross-Site Scripting (XSS)

If user-supplied text is inserted into a page without escaping, an attacker can inject `<script>` tags that run in other users' browsers. Always escape output and prefer safe templating APIs (like `textContent` over `innerHTML`) when working with untrusted data.

OWASP — Cross Site Scripting Prevention Cheat Sheet

https://cheatsheetseries.owasp.org/cheatsheets/Cross_Site_Scripting_Prevention_Cheat_Sheet.html

Industry-standard, vendor-neutral guidance for preventing XSS.

MDN — Website security

https://developer.mozilla.org/en-US/docs/Learn/Server-side/First_steps/Website_security

A web-developer-focused overview of common vulnerabilities.



- ☐ Use one meaningful, unique title per page.
- ☐ Use clear headings and a logical semantic structure.
- ☐ Write descriptive link text instead of “click here.”
- ☐ Use appropriate, descriptive alt text on informative images.
- ☐ Write genuinely useful, original content for your actual audience.
- ☐ Use a meta description that accurately summarizes the page.
- ☐ Use canonical URLs to avoid duplicate-content issues.
- ☐ Add accurate structured data (schema.org) where it applies.

```
seo-head.html HTML

<head>
  <title>HTML Forms &mdash; Complete Guide</title>
  <meta name="description"
        content="Learn HTML forms and validation with examples.">
  <link rel="canonical" href="https://example.com/html/forms">
</head>
```



Good HTML helps, it doesn't guarantee

Semantic, well-structured HTML helps search engines parse and understand your content correctly, but it does not by itself guarantee any particular ranking.

Google Search Central — SEO Starter Guide

<https://developers.google.com/search/docs/fundamentals/seo-starter-guide>

Google's own official guide to search-friendly site structure.

schema.org

<https://schema.org/>

The shared vocabulary for structured data used by all major search engines.



```
integration.html HTML

<!doctype html>
<html lang="en">
<head>
  <link rel="stylesheet" href="styles.css">
</head>
<body>
  <button id="hello">Say hello</button>
  <script src="app.js" defer></script>
</body>
</html>
```

Learning model: **HTML** = structure/meaning, **CSS** = presentation, **JavaScript** = behavior. Build meaningful HTML first, then style it, then add scripted behavior only where it's genuinely needed.

Progressive Enhancement



1. HTML

Useful, readable, functional content and controls that work with zero CSS or JS.



2. CSS

Layer on visual design, layout, and responsiveness.



3. JavaScript

Enhance with interactivity — but the page should still work reasonably if it fails to load.

MDN — How CSS is structured

https://developer.mozilla.org/en-US/docs/Learn/CSS/First_steps/How_CSS_is_structured

How stylesheets attach to and target HTML.

MDN — JavaScript basics

https://developer.mozilla.org/en-US/docs/Learn/JavaScript/First_steps/What_is_JavaScript

Where JavaScript fits alongside HTML and CSS.



API	Role
DOM	JavaScript's live representation of the HTML document
Fetch API	Making network requests from the browser
Web Storage	localStorage / sessionStorage for client-side key-value data
Geolocation API	Requesting the user's location, with permission
Canvas API	A scriptable, pixel-based drawing surface
WebSocket API	Persistent, bidirectional client-server communication
History API	Reading and manipulating browser session history
Clipboard API	Reading from and writing to the system clipboard
Web Workers	Running JavaScript on a background thread
Service Workers	A network/cache proxy enabling offline support and push
Intersection Observer	Efficiently detecting when elements enter the viewport



Not all HTML tags, not all APIs

These are commonly learned alongside HTML, but most are JavaScript APIs rather than markup. canvas, for example, is a real HTML element, but everything drawn inside it happens through the Canvas JavaScript API, not through HTML attributes.

[MDN — Web APIs index](https://developer.mozilla.org/en-US/docs/Web/API)

<https://developer.mozilla.org/en-US/docs/Web/API>

The complete, official catalogue of every browser Web API.



Old Element	Modern Replacement
<code>font</code> , <code>center</code>	CSS (font-family/color, text-align/margin)
<code>strike</code>	<code>s</code> or <code>del</code> , depending on meaning
<code>tt</code>	<code>code</code> or another appropriate semantic element
<code>frameset</code> / <code>frame</code>	Modern CSS layout; <code>iframe</code> only for real embedding
<code>marquee</code> , <code>blink</code>	Avoid entirely; use accessible CSS/JS if truly required
<code>acronym</code>	<code>abbr</code>
<code>big</code>	CSS font-size



Why these were dropped

Early HTML mixed presentation into markup. These elements make content harder to restyle, harder to make accessible, and harder to maintain — CSS handles all of their original jobs far more flexibly.

MDN — Obsolete and deprecated HTML elements

https://developer.mozilla.org/en-US/docs/Web/HTML/Element#obsolete_and_deprecated_elements

Official list of elements no longer recommended for use.



Common Mistakes

The errors that show up again and again in real projects

Mistake	Better Approach
Missing doctype	Always start with <code><!doctype html></code>
Missing lang	Set <code>lang</code> on the html element
Image without alt	Provide an appropriate alt (or alt="" if decorative)
Input without a label	Pair every input with a real label via for/id
A div for everything	Use semantic elements where they fit
A table for layout	Use CSS Flexbox/Grid for layout
br used for spacing	Use CSS margin/padding instead
Duplicate IDs	Keep every id unique within the document
Trusting client-side validation only	Always validate and authorize server-side
Secrets in HTML	Keep all secrets strictly server-side
Unoptimized, oversized images	Compress and serve responsive image candidates



Run the W3C validator

Paste your HTML into the official Nu Html Checker regularly during development — it catches most of the mistakes above automatically, before they reach production.

W3C Nu Html Checker (validator)

<https://validator.w3.org/nu/>

The official, up-to-date HTML validator used across the industry.



portfolio-complete.html

HTML

```
<!doctype html>
<html lang="en">
<head>
  <meta charset="utf-8">
  <meta name="viewport" content="width=device-width, initial-scale=1">
  <meta name="description" content="Student portfolio">
  <title>Student Portfolio</title>
</head>
<body>
<header>
  <h1>Student Portfolio</h1>
  <nav aria-label="Main navigation">
    <a href="#about">About</a>
    <a href="#projects">Projects</a>
    <a href="#contact">Contact</a>
  </nav>
</header>
<main>
  <section id="about">
    <h2>About</h2>
    <p>I am an IT student learning web development.</p>
  </section>
  <section id="projects">
    <h2>Projects</h2>
    <article>
      <h3>Portfolio Website</h3>
      <p>A semantic HTML project.</p>
    </article>
  </section>
  <section id="contact">
    <h2>Contact</h2>
    <form action="/contact" method="post">
      <label for="name">Name</label>
      <input id="name" name="name" required>
      <label for="email">Email</label>
      <input id="email" name="email" type="email" required>
      <label for="message">Message</label>
      <textarea id="message" name="message" rows="5" required></textarea>
      <button type="submit">Send</button>
    </form>
  </section>
</main>
<footer><p>&copy; 2026 Student Portfolio</p></footer>
</body>
</html>
```



Type this exact file, open it with zero CSS, and confirm every landmark and form control already works and makes sense read aloud. Only then start layering on CSS, then JavaScript.



Personal Profile

Headings, paragraphs, an image, links, and lists.

Portfolio Site

Semantic layout, navigation, and article elements.

Resume Page

Lists, links, and contact information.

Registration Form

Labels, varied input types, and validation attributes.

Product Page

Images, sections, and a specifications table.

Blog Post

article, figure, blockquote, and time elements.

Course Catalogue Site

Navigation, sections, tables, and forms together.

Accessible Contact Page

Labels, landmarks, and keyboard-only testing.

Media Page

Audio, video, and captions.

Full Capstone Portfolio

Combine every concept from this course into one site.



Recommended sequence

HTML only → then CSS → then JavaScript. Building each project in this order keeps each technology's responsibility crystal clear, and makes debugging far easier.



DOCUMENT SKELETON

```

skeleton.html HTML
<!doctype html>
<html lang="en">
<head>
  <meta charset="utf-8">
  <meta name="viewport"
content="width=device-width, initial-
scale=1">
  <title>Page</title>
</head>
<body></body>
</html>
```

STRUCTURE

header · nav · main · section · article · aside · footer

TEXT

h1–h6 · p · strong · em · mark · small · code · kbd ·
blockquote · q · pre · br · hr · del · ins · sub · sup

MEDIA

a · img · picture · source · audio · video · track · iframe

LISTS

ul · ol · li · dl · dt · dd

TABLES

table · caption · thead · tbody · tfoot · tr · th · td · colgroup
· col

FORMS

form · label · input · textarea · select · option · optgroup ·
button · fieldset · legend · datalist · output

HIGH-VALUE ATTRIBUTES

id · class · lang · dir · title · hidden · tabindex · data-* · href
· src · alt · width · height · name · value · type · action ·
method · target · rel · required · placeholder · disabled ·
readonly · checked · selected · min · max · minlength ·
maxlength · pattern · autocomplete · accept

Q1 What is HTML?

A HyperText Markup Language — the standard markup language for structuring web content.

Q2 Is HTML a programming language?

A No. It is a markup language; it has no variables, loops, or functions.

Q3 What is the DOM?

A A programmatic, tree-shaped representation of the document that JavaScript and CSS interact with.

Q4 Why does the doctype matter?

A It requests standards mode and avoids legacy quirks-mode rendering bugs.

Q5 Why set lang on the html element?

A It helps browsers, assistive technology, and translation tools interpret content correctly.

Q6 Why is alt text important?

A It gives images an appropriate, accessible text alternative for screen readers and failed loads.

Q7 Why pair a label with an input?

A It gives the control a clear accessible name and a larger clickable/tappable target.

Q8 GET vs POST?

A GET commonly places data in the URL query string; POST commonly sends it in the request body.

Q9 What does “semantic HTML” mean?

A Choosing elements according to their real meaning, not their default visual appearance.

Q10 Is a hidden input secure?

A No — the client can freely view and modify its value before submitting.

Q11 Is client-side validation ever enough on its own?

A No. It improves user experience, but the server must always re-validate and authorize.

Q12 What is srcset for?

A It offers multiple image file candidates so the browser can pick the best one for the device.

Q13 What are tables meant for?

A Tabular data with real rows and columns — never for page layout.

Q14 Link vs button — what's the difference?

A A link navigates to a new location; a button performs an action on the current page.

Q15 What does the required attribute do?

A It adds a built-in constraint requiring the field to have a value before submission.

Q16 What is an iframe?

A An embedded browsing context that displays another HTML document inside the current page.

Q17 What is progressive enhancement?

A Useful HTML first, layered CSS second, and JavaScript enhancement third.

Q18 Why avoid deprecated elements like font and center?

A Modern semantic HTML plus CSS is more flexible, accessible, and maintainable.

Q19 What does accessibility mean in practice?

A Making content genuinely usable by people with a wide range of abilities and assistive technologies.

Q20 What are the three basic layers of front-end development?

A HTML for structure, CSS for presentation, and JavaScript for behavior.

- ☐ Explain element, tag, attribute, content, nesting, and void elements.
- ☐ Write a complete HTML document from memory, with no reference.
- ☐ Use metadata (charset, viewport, description) correctly.
- ☐ Use global attributes correctly (id, class, lang, data-*, etc).
- ☐ Create a meaningful, unbroken heading hierarchy.
- ☐ Use semantic text elements instead of styled generic tags.
- ☐ Create absolute, relative, root-relative, and fragment links.
- ☐ Use images with correct, meaningful alt text.
- ☐ Use srcset/sizes/picture for responsive images.
- ☐ Embed audio/video with captions and fallbacks.
- ☐ Build accessible lists and tables with proper scope.
- ☐ Create forms with labels and the correct input types for each field.
- ☐ Use validation constraints correctly, and never trust them alone.
- ☐ Use fieldset/legend to group related form controls.
- ☐ Build fully semantic page layouts.
- ☐ Know precisely when div/span are the appropriate choice.
- ☐ Understand the risks of iframe and third-party embedded content.
- ☐ Use entities and relative/absolute paths correctly.
- ☐ Apply accessibility fundamentals throughout every project.
- ☐ Understand how responsive HTML and CSS divide responsibility.
- ☐ Keep all secrets strictly out of client-side HTML.
- ☐ Always validate and authorize on the server.
- ☐ Use SEO-friendly, crawlable structure.
- ☐ Connect HTML cleanly to CSS and JavaScript.
- ☐ Know the major browser and Web APIs by name and purpose.
- ☐ Recognize and avoid obsolete HTML elements.
- ☐ Build at least three complete projects from this course, unaided.

Final mental model

HTML gives structure and meaning → CSS gives presentation → JavaScript gives behavior. Strong HTML is semantic, accessible, maintainable, and understandable long before any styling is added.

Next learning path: **CSS** → selectors/cascade → box model → Flexbox → Grid → responsive design → animation; then **JavaScript** → DOM → events → forms → APIs → full applications.



HTML5 Timeline & History

How markup got here, in nine milestones



1991

Tim Berners-Lee publishes the first description of HTML at CERN.



1995

HTML 2.0 is published as an IETF RFC, the first formal specification.



1997–1999

HTML 3.2 and HTML 4.01 are released as W3C Recommendations, adding tables, forms, and scripting hooks.



2000

XHTML 1.0 reformulates HTML as strict XML; an ambitious XHTML 2.0 effort later stalls.



2004

Mozilla, Apple, and Opera engineers form WHATWG to continue evolving HTML pragmatically.



2008

The first HTML5 Working Draft is published, introducing semantic elements, audio/video, and canvas.



2014

HTML5 becomes an official W3C Recommendation.



2019

The W3C and WHATWG agree the WHATWG Living Standard is the single authoritative source for HTML.



Today

HTML has no more version numbers — the Living Standard is continuously updated as browsers implement new features.

W3C — A History of HTML

<https://www.w3.org/wiki/HTML/History>

The W3C's own account of HTML's evolution.

WHATWG — FAQ

<https://whatwg.org/faq>

Why the Living Standard model replaced versioned HTML specs.



Official Resources & Further Reading

Bookmark these — they are the sources this course draws from

Specifications

WHATWG HTML Living Standard

<https://html.spec.whatwg.org/multipage/>

The single authoritative, continuously updated HTML specification.

W3C — Web Standards

<https://www.w3.org/standards/>

The World Wide Web Consortium's index of official web standards.

Learning & Reference

MDN Web Docs — HTML

<https://developer.mozilla.org/en-US/docs/Web/HTML>

The most widely used HTML reference and tutorial hub among professional developers.

web.dev by Google

<https://web.dev/>

Structured, modern learning paths for HTML, CSS, JavaScript, and performance.

freeCodeCamp

<https://www.freecodecamp.org/learn>

Free, project-based curriculum covering HTML through full-stack development.

Accessibility

W3C Web Accessibility Initiative (WAI)

<https://www.w3.org/WAI/>

Official accessibility guidelines, including WCAG.

WebAIM

<https://webaim.org/>

Practical accessibility testing tools and guides, including contrast checkers.

Tools

W3C Nu Html Checker

<https://validator.w3.org/nu/>

Validate any HTML document against the official specification.

Can I Use

<https://caniuse.com/>

Real-time browser support tables for HTML, CSS, and JS features.



A note on authenticity

Every link in this guide points to the primary, official source for that topic — the specification body itself (WHATWG/W3C), the browser vendor's own documentation (MDN, web.dev), or the recognized authority for that domain (OWASP for security, WAI for accessibility, Google Search Central for SEO). Bookmark this page as your jumping-off point.

You've completed the course.

Structure with HTML. Style with CSS. Bring it to life with JavaScript. Build accessibly, securely, and semantically — every single time.

HTML • CSS • JAVASCRIPT